

EE 475 Introduction to Image Processing

Homework 02

Boran Aybak Kilic
Student ID: 2022401207

1 a) Creating a Circular Mask

To create a mask in the shape of a disk, we define the mask by selecting all points whose Euclidean distance from the center is less than or equal to a specified radius.

For implementation in MATLAB, a 2D coordinate grid with the same dimensions as the input image is created using `[X, Y] = meshgrid(1:N, 1:M);`, where 'M' and 'N' are the image's height (rows) and width (columns), respectively. The output matrices 'X' and 'Y' are the same size as the input image and hold the x-coordinate and y-coordinate of each pixel.

Next, the Euclidean distance of each pixel from the center pixel is calculated in a single, element-wise operation: `distance = sqrt((X-cx).^2 + (Y-cy).^2);`. Here, `cx = floor(N/2);` and `cy = floor(M/2);` are the coordinates of the center pixel.

A logical (binary) mask is then created by thresholding this distance matrix: `mask_logical = (distance <= radius);`. Finally, the masked image is obtained by first converting the logical mask to `uint8` and then performing an element-wise multiplication with the original image.

```
1 chro = imread("Chronometer.tif");
2 [M, N] = size(chro);
3
4 % Create a coordinate grid
5 [X, Y] = meshgrid(1:N, 1:M);
6
7 % Find center coordinates
8 cx = floor(N/2);
9 cy = floor(M/2);
10
11 % Calculate Euclidean distance of every pixel from the center
12 distance = sqrt((X-cx).^2 + (Y-cy).^2);
13 radius = 500;
14 mask_logical = (distance <= radius); % Creates a logical (true/false) mask
15
16 % Convert logical mask to uint8 (1s and 0s) for multiplication
17 masked_image = chro .* uint8(mask_logical);
18
19 % Display the results
20 figure;
21 t = tiledlayout(1, 3, 'Padding', 'compact', 'TileSpacing', 'compact');
22 nexttile
23 imshow(chro);
24 title('Original Image', 'FontSize', 18); axis on;
25 nexttile
26 imshow(mask_logical); % imshow correctly displays logical as B&W
27 title('Mask', 'FontSize', 18); axis on;
28 nexttile
29 imshow(masked_image);
30 title('Masked Image', 'FontSize', 18); axis on;
```

Listing 1: Code to create a circular mask based on Euclidean distance.

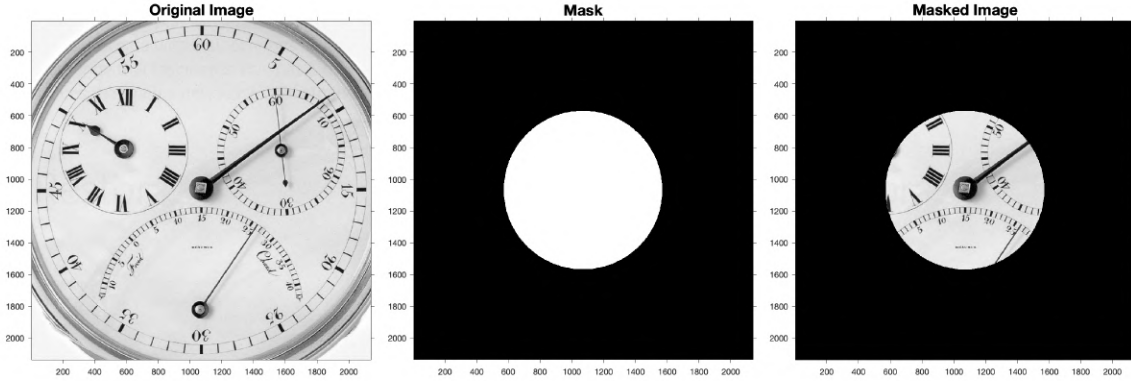


Figure 1: The chronometer, the mask, and the result after applying the mask.

2 b) Mathematical Description of Circular Mask

As outlined previously, the mask $M(x, y)$ is defined by selecting all pixels whose Euclidean distance from the center is less than or equal to a given radius r :

$$M(x, y) = \begin{cases} 1 & \text{if } \sqrt{(x - c_x)^2 + (y - c_y)^2} \leq r \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where $c_x = \lfloor N/2 \rfloor$ and $c_y = \lfloor M/2 \rfloor$ are the coordinates of the center pixel, and $[M, N]$ are the dimensions of the image.

3 c) Generalized Mask Function

Generalizing the previous method, we can create a mask consisting of pixels whose "distance" to the center is less than a given radius. This "distance" does not have to be Euclidean; by substituting other distance metrics, we can generate masks of different shapes. These alternative metrics are often used in morphological operations or connectivity analysis.

We will define two additional distance metrics besides the Euclidean (L_2) norm:

- **D_4 distance / City-block Metric (L_1 -norm):** The City-block distance between two points $p = (x_1, y_1)$ and $q = (x_2, y_2)$ is defined as the sum of the absolute differences of their coordinates:

$$D_4(p, q) = |x_1 - x_2| + |y_1 - y_2|. \quad (2)$$

This metric is also called the D_4 distance because it measures the path length between pixels when only 4-connected neighbors (up, down, left, right) are considered. It generates iso-distance contours (lines of equal distance) in the shape of a **diamond**.

- **D_8 distance / Chessboard Metric (L_∞ -norm):** The Chessboard distance between two points $p = (x_1, y_1)$ and $q = (x_2, y_2)$ is defined as the maximum of the absolute differences of their coordinates:

$$D_8(p, q) = \max\{|x_1 - x_2|, |y_1 - y_2|\}. \quad (3)$$

This metric is called the D_8 distance because it measures the path length when 8-connected neighbors (including diagonals) are considered, just like a king's move in chess. It generates iso-distance contours in the shape of an axis-aligned **square**.

The new, generalized mask function 'createMask' is implemented to accept a 'metric_type' string as an argument, allowing it to generate a mask using any of the defined metrics.

```

1 function mask = createMask(image, radius, metric_type)
2     % createMask: Generates a logical mask for an image
3     %
4     % Inputs:
5     %   image      - The 2D input image (to get dimensions)
6     %   radius     - The distance threshold
7     %   metric_type - 'euclidean', 'cityblock', or 'chessboard'
8     %
9     % Output:
10    %   mask        - A logical (binary) matrix of the same size as image
11
12    [M, N] = size(image);
13    [X, Y] = meshgrid(1:N, 1:M);
14    cx = floor(N/2);
15    cy = floor(M/2);
16
17    % Calculate distance based on the chosen metric
18    switch lower(metric_type)
19        case 'euclidean'
20            % L2-norm
21            distance = sqrt((X - cx).^2 + (Y - cy).^2);
22        case 'cityblock'
23            % L1-norm
24            distance = abs(X-cx) + abs(Y-cy);
25        case 'chessboard'
26            % L-infinity-norm
27            distance = max(abs(X-cx), abs(Y-cy));
28        otherwise
29            error('Unknown metric type. Use "euclidean", "cityblock", or "chessboard".');
30    end
31
32    % Create the final logical mask.
33    % Returning a logical mask is more memory-efficient and flexible
34    % than returning uint8. The caller can cast it as needed.
35    mask = (distance <= radius);
36 end

```

Listing 2: Code to create a general mask based on a given distance metric.

4 d) Demonstration of City-block Metric

As expected, the mask created using the city-block metric has a *diamond shape*, as seen in Fig. 2. As an additional demonstration, a mask was also constructed using the chessboard metric, which correctly yields a *square shape*, as can be observed in Fig. 3.

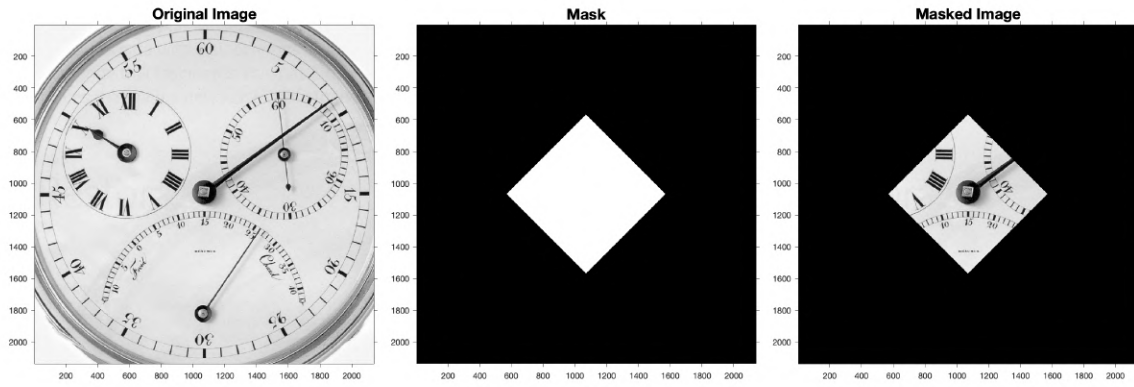


Figure 2: Image, mask, and the resulting masked image for the **city-block** metric.

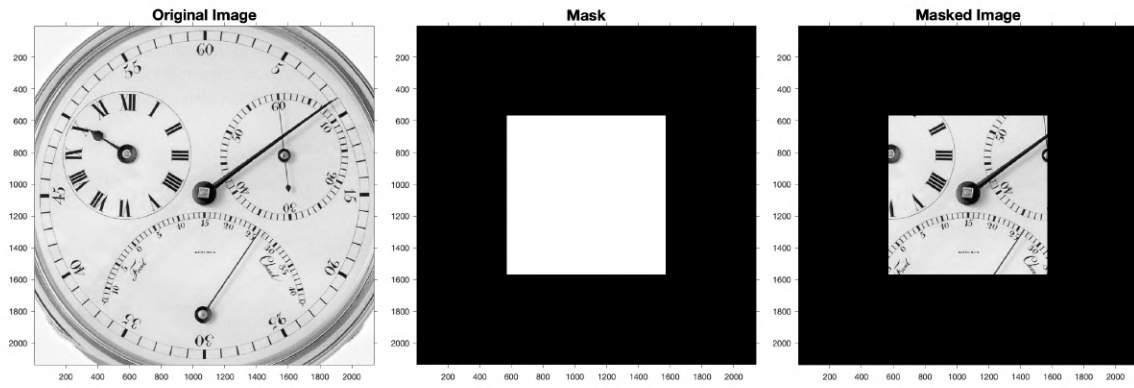


Figure 3: Image, mask, and the resulting masked image for the **chessboard** metric.